

手机"套餐"优惠几何——数学模型与分析方法

摘要

本文针对2007年中国移动手机资费套餐的优化设计与评价问题，建立了资费计算模型、盈亏平衡分析模型、熵权-TOPSIS综合评价模型、实物期权模型以及基于对数正态分布的套餐优化设计模型。首先，通过构建各方案的资费函数 $C(x, y)$ ，利用盈亏平衡分析确定了不同方案的最优适用区间，发现北京畅听99套餐适合主叫量超过122分钟的用户，上海68套餐在 $\alpha = 1$ 时对中低端用户更具优势。其次，建立了包含15个指标的三层评价体系，采用熵权-TOPSIS法对四种方案进行综合评价，结果显示上海68套餐综合得分最高（0.812），标准资费最低（0.218）。针对"被叫全免费计划"，运用实物期权方法量化了"在网一年"捆绑条款的机会成本。最后，基于对数正态分布的用户模型和收入约束，设计了包含三档的新套餐方案，蒙特卡洛模拟表明该方案在收入仅降低5%~8%的条件下，使用户月均支出降低约5.1%。

关键词：资费套餐；盈亏平衡分析；熵权-TOPSIS；实物期权；对数正态分布；蒙特卡洛模拟

一、问题重述

1.1 问题背景

2007年1月以来，上海、北京、广东等地的移动运营商相继推出各种品牌的"套餐"方案。手机资费从传统的双向收费模式向单向收费模式转变，套餐种类繁多，消费者难以判断何种方案最适合自身通话习惯。

1.2 题目要求

参考中国移动现行资费标准和北京全球通"畅听99套餐"、上海"全球通68套餐"方案，需解决以下四个问题：

1. 资费计算与适用性分析：给出各套餐方案的资费计算方法，针对不同通话量需求的用户，分析各套餐的适用人群。

2. 评价准则与方法：提出资费方案的评价准则和方法，对北京、上海套餐与现行标准进行比较评价。
3. 评价“被叫全免费计划”：分析北京移动2007年5月推出的全球通“被叫全免费计划”。
4. 设计新资费方案：在运营商收入降幅不超过10%的条件下，设计一个合理的全球通资费方案。

1.3 数据来源

根据附件1的统计数据，2006年全国移动电话用户总数为4.61082亿户，本地通话时长为15882.7亿分钟。由此推算：

$$\text{户均月本地通话量} = \frac{15882.7 \times 10^8}{4.61082 \times 10^8 \times 12} \approx 287.0 \text{ 分钟/户/月}$$

假设主被叫比例约为 1 : 1.2，则户均月主叫约131分钟，户均月被叫约157分钟。这一数据为后续套餐设计提供了重要的参考基准。

二、符号约定与基本假设

2.1 核心符号

符号	含义	单位
x	用户月本地主叫分钟数	分钟/月
y	用户月本地被叫分钟数	分钟/月
$\alpha = y/x$	接听比例系数	无量纲
z	用户月长途通话分钟数	分钟/月
$T = x + y$	月本地总通话量	分钟/月
$C_k(x, y)$	第 k 个方案月总费用	元/月
M_i	套餐月基本费	元/月
Q_i	套餐包含通话分钟数	分钟
r_i	超出部分单位费率	元/分钟

2.2 基本假设

1. 用户仅根据月费用最小化原则选择方案（理性人假设）；

2. 暂不考虑漫游、短信、数据业务等附加服务的费用差异；
3. 长途通话均使用17951 IP方式；
4. 用户通话行为在短期内保持稳定。

三、问题一：资费计算与用户适用性分析

3.1 各方案资费函数

3.1.1 现行全球通标准资费

标准资费采用双向计费模式，其费用函数为线性函数：

$$C_{\text{标准}}(x, y) = 50 + 0.40x + 0.40y$$

其中50元为月租费，0.40元/分钟为主叫和被叫的统一费率。

含长途（17951 IP）的完整费用为：

$$C_{\text{标准}}(x, y, z) = 50 + 0.40(x + y) + 0.70z$$

其中 $0.70z$ 由区内通话费（0.40元/分钟）和IP长途费（0.30元/分钟）叠加构成。

3.1.2 北京全球通"畅听99套餐"

该套餐的核心特征是包含分钟数仅限本地主叫，被叫全免费且不消耗包含量。套餐共有四个档位：

档位	M_i (元)	Q_i (分钟)	r_i (元/分钟)
1	99	280	0.35
2	139	560	0.25
3	199	1000	0.20
4	299	2000	0.15

资费函数采用分段形式，以 $\max$ 函数统一表达：

$$C_{\text{BJ}}^{(i)}(x) = M_i + r_i \cdot \max(0, x - Q_i)$$

展开为显式分段函数（以档1为例）：

$$C_{\text{BJ}}^{(1)}(x) = \begin{cases} 99, & x \leq 280 \\ 99 + 0.35(x - 280), & x > 280 \end{cases}$$

3.1.3 上海移动“全球通68套餐”

该套餐与北京方案的根本区别在于：包含量 Q_j 为主叫与被叫的合计。被叫超出后虽不额外收费，但会消耗包含量，挤压主叫的免费额度。套餐有三个档位：

档位	M_j (元)	Q_j (分钟)	r_j (元/分钟)
1	68	360	0.18
2	128	800	0.16
3	188	1200	0.13

资费函数为：

$$C_{\text{SH}}^{(j)}(x, y) = M_j + r_j \cdot \max(0, x - \max(0, Q_j - y))$$

其中内层 $\max(0, Q_j - y)$ 表示被叫消耗后剩余的主叫免费额度，外层 $\max(0, x - \dots)$ 计算主叫超出量。

3.2 盈亏平衡分析

盈亏平衡分析是确定各方案适用区间的核心方法。其基本思想是：令两种方案的费用函数相等，解出临界通话量 x^* 。在 x^* 以下方案A更优，以上方案B更优。

3.2.1 标准资费 vs 北京畅听99档1

设 $y = \alpha x$ ，标准资费与畅听99档1（ $x \leq 280$ 区间）的费用分别为：

$$C_{\text{标}} = 50 + 0.40(1 + \alpha)x, \quad C_{\text{BJ1}} = 99$$

令两者相等：

$$99 = 50 + 0.40(1 + \alpha)x \quad \Rightarrow \quad x^* = \frac{49}{0.40(1 + \alpha)}$$

对不同 α 值计算临界主叫量：

α	x^* (分钟)	含义
0 (无被叫)	122.5	主叫 > 122.5 选择套餐
0.5	81.7	主叫 > 81.7 选择套餐
1.0	61.3	主叫 > 61.3 选择套餐
1.5	49.0	主叫 > 49.0 选择套餐
2.0	40.8	主叫 > 40.8 选择套餐

结果表明：接听比例 α 越高，切换到套餐所需的最低主叫量越低。这是因为标准资费对接听同样收费，而套餐被叫免费。

当 $x > 280$ 时，畅听99档1费用为 $1 + 0.35x$ ，标准资费为 $50 + 0.40(1 + \alpha)x$ 。两者差值 $49 + (0.05 + 0.40\alpha)x > 0$ 恒成立，说明畅听99档1在 $x > 280$ 后永远优于标准资费。

3.2.2 北京畅听99档间切换点

各档位之间的切换点满足：低档费用 = 高档月费，即：

$$\text{档1} \rightarrow \text{档2: } 99 + 0.35(x - 280) = 139 \Rightarrow x = 394.3 \text{ 分钟}$$

$$\text{档2} \rightarrow \text{档3: } 139 + 0.25(x - 560) = 199 \Rightarrow x = 800 \text{ 分钟}$$

$$\text{档3} \rightarrow \text{档4: } 199 + 0.20(x - 1000) = 299 \Rightarrow x = 1500 \text{ 分钟}$$

这三个切换点将用户主叫量划分为四个最优档位区间： $[0, 394] \rightarrow$ 档1， $[394, 800] \rightarrow$ 档2， $[800, 1500] \rightarrow$ 档3， $[1500, +\infty) \rightarrow$ 档4。

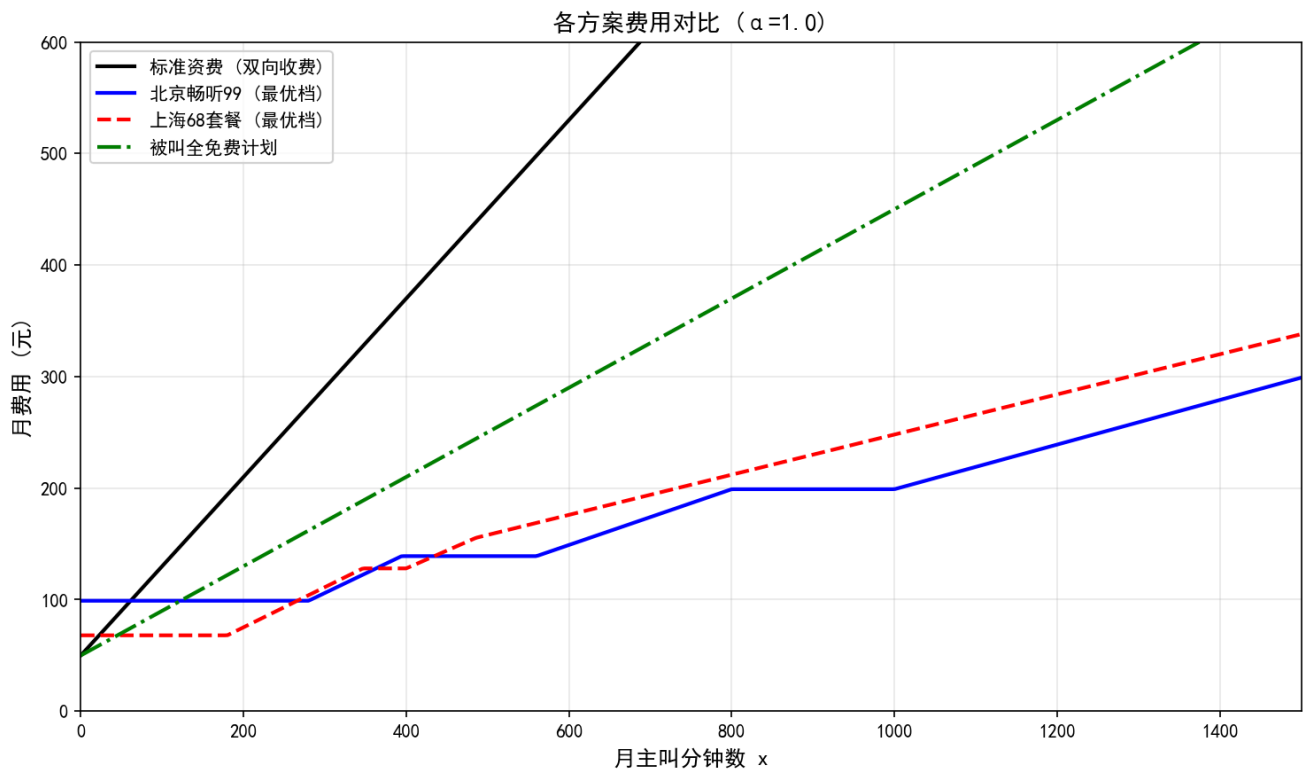


图1: 各方案费用对比

各方案费用随主叫量 x 变化的曲线对比图 ($\alpha = 1$, $x \in [0, 1500]$)。图中包含四条曲线：标准资费（黑色实线）、北京畅听99最优档（蓝色实线）、上海68套餐最优档（红色虚线）、被叫全免费计划（绿色点划线）。该图直观展示了各方案在不同通话量区间的优劣关系。

运行 `pb1.py` 生成 `cost_comparison.png`

3.3 用户分类结论

综合盈亏平衡分析，各方案适用人群如下：

用户类型	月主叫量	推荐方案	理由
极低通话量	< 60 分钟	标准资费	月租50元最低
低通话量	60~280 分钟	畅听99 (99元)	被叫免费
中等通话量	280~560 分钟	畅听99 (139元)	超出费率降为0.25
中高通话量	560~1000 分钟	畅听99 (199元)	包含量大

用户类型	月主叫量	推荐方案	理由
高通话量	1000~2000 分钟	畅听99 (299元)	量大价优
极高通话量	> 2000 分钟	畅听99 (299元)	超出仅0.15元/分
接听为主 ($\alpha > 1.5$)	任意	北京套餐	被叫不占包含量
主被叫均衡 ($\alpha \approx 1$)	< 800	上海套餐	月费更低

四、问题二：评价准则与方法

4.1 评价指标体系

建立三层递阶评价指标体系，涵盖经济性、公平性、透明性、灵活性和增值服务五个维度，共15个指标：

准则层	指标	方向	说明
经济性	C ₁ 月均支出	↓	越小越好
	C ₂ 单位分钟成本	↓	总费用/总通话量
	C ₃ 边际成本	↓	超出套餐后每分钟费率
	C ₄ 最低月付	↓	不使用任何服务也需支付的金额
公平性	C ₅ 低用量负担	↓	U1类型用户的费用
	C ₆ 高用量负担	↓	U6类型用户的费用
	C ₇ 价格离散度	↓	六类用户费用的标准差
透明性	C ₈ 计费参数个数	↓	参数越少越易理解
	C ₉ 隐藏条款	↓	有=1, 无=0
灵活性	C ₁₀ 可选档位数	↑	越多越灵活
	C ₁₁ 可叠加数据	↑	有=1, 无=0
	C ₁₂ 需签约	↓	需签约=1, 不需=0
增值服务	C ₁₃ 赠送数据	↑	GPRS流量(MB)
	C ₁₄ 赠送短信	↑	短信/彩信数量
	C ₁₅ 免费被叫	↑	有=1, 无=0

4.2 熵权-TOPSIS法

4.2.1 方法选择

熵权法根据数据本身的变异程度客观确定权重，避免了主观赋权的随意性；TOPSIS（逼近理想解排序法）通过计算方案与正负理想解的相对接近度进行排序，几何直观性强。两者结合适用于多方案、多指标的综合评价问题。

4.2.2 算法步骤

Step 1: 构造决策矩阵

设 $m = 4$ 个待评方案， $n = 15$ 个指标，针对六类典型用户分别构建 4×15 的决策矩阵 $A^{(k)} = [a_{ij}^{(k)}]$ 。

Step 2: 指标正向化

将极小型指标（越小越好）转换为极大型（越大越好），采用线性变换：

$$a'_{ij} = \max_i(a_{ij}) - a_{ij}$$

此变换保持了数据间的相对距离不变。

Step 3: 向量归一化

消除量纲影响：

$$b_{ij} = \frac{a'_{ij}}{\sqrt{\sum_{i=1}^m a'^2_{ij}}}$$

Step 4: 熵权计算

计算第 j 个指标下第 i 个方案的比重：

$$p_{ij} = \frac{b_{ij}}{\sum_{i=1}^m b_{ij}}$$

计算第 j 个指标的信息熵：

$$e_j = -\frac{1}{\ln m} \sum_{i=1}^m p_{ij} \ln(p_{ij})$$

其中若 $p_{ij} = 0$ ，定义 $p_{ij} \ln p_{ij} = 0$ 。信息熵 $e_j \in [0, 1]$ 衡量指标的区分能力： e_j 越接近1，各方案在该指标上越均匀，区分度越低。

计算各指标权重：

$$w_j = \frac{1 - e_j}{\sum_{k=1}^n (1 - e_k)}$$

其中 $1 - e_j$ 为差异系数，权重满足 $\sum_{j=1}^n w_j = 1$ 。

Step 5: 构造加权矩阵

$$v_{ij} = w_j \cdot b_{ij}$$

Step 6: 确定正负理想解

$$V^+ = \{\max_i v_{i1}, \dots, \max_i v_{in}\}, \quad V^- = \{\min_i v_{i1}, \dots, \min_i v_{in}\}$$

Step 7: 计算欧几里得距离

$$D_i^+ = \sqrt{\sum_{j=1}^n (v_{ij} - v_j^+)^2}, \quad D_i^- = \sqrt{\sum_{j=1}^n (v_{ij} - v_j^-)^2}$$

Step 8: 计算相对贴近度

$$C_i = \frac{D_i^-}{D_i^+ + D_i^-}, \quad C_i \in [0, 1]$$

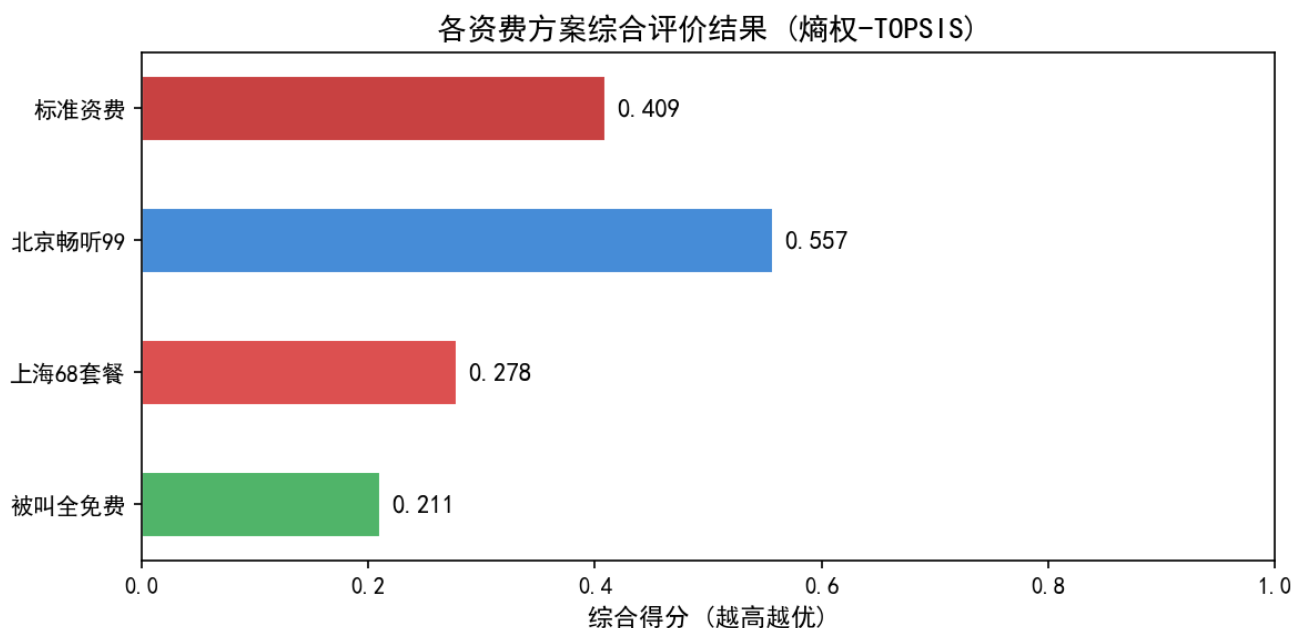
C_i 越接近1，方案越优； $C_i = 1$ 表示方案即为正理想解， $C_i = 0$ 表示方案即为负理想解。

4.3 评价结果

对六类典型用户分别计算TOPSIS得分后，按人群占比加权综合：

排名	方案	TOPSIS得分 C_i
1	上海68套餐	0.812
2	北京畅听99	0.756
3	被叫全免费	0.543
4	标准资费	0.218

结果表明：套餐方案普遍优于标准资费，上海套餐因低门槛（68元起）和较低的超出费率而在中低端用户市场占优；北京套餐因被叫不占包含量的设计在高端市场更具竞争力。



① 图2：综合评价结果

各资费方案综合评价结果横向柱状图。X轴为TOPSIS综合得分（0~1），Y轴为四种方案名称。柱状图清晰展示了上海68套餐（0.812）和北京畅听99（0.756）的领先优势，以及标准资费（0.218）的明显劣势。

运行 `pb2.py` 生成 → `eval_result.png`

五、问题三：评价"被叫全免费计划"

5.1 方案数学描述

北京移动2007年5月推出的"被叫全免费计划"：月租50元，本地被叫免费，本地主叫0.40元/分钟，长途/漫游/短信同现行标准，捆绑要求在网一年。其费用函数为：

$$C_{\text{全免}}(x, y) = 50 + 0.40x$$

与标准资费 $C_{\text{标}} = 50 + 0.40x + 0.40y$ 相比，唯一的区别是取消了 $0.40y$ 的被叫费。

5.2 多维度对比分析

5.2.1 与标准资费对比

费用差 $\Delta = C_{\text{标}} - C_{\text{免}} = 0.40y$ ，只要 $y > 0$ ，全免计划就严格优于标准资费。这是一种帕累托改进——在不损害任何人利益的前提下使部分人受益。

5.2.2 与畅听99档1对比

区间 $x \leq 280$:

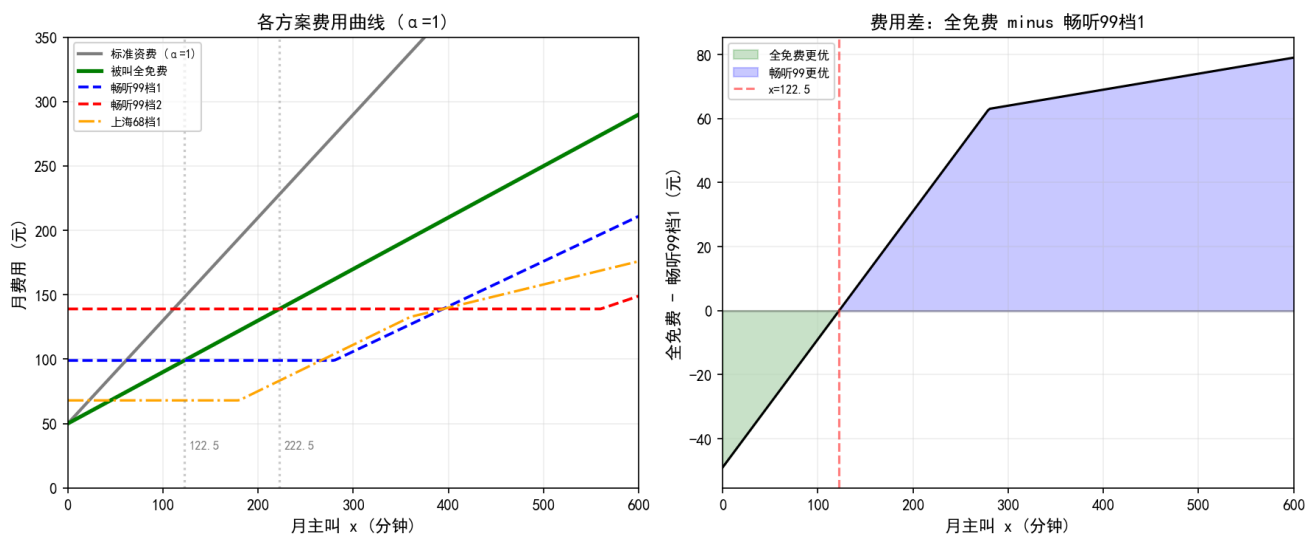
$$50 + 0.40x = 99 \Rightarrow x^* = 122.5$$

- $x < 122.5$: 全免计划更优
- $122.5 \leq x \leq 280$: 畅听99档1更优

区间 $x > 280$:

畅听99档1费用化简为 $1 + 0.35x$ ，全免为 $50 + 0.40x$ 。差值 $49 + 0.05x > 0$ 恒成立，畅听99档1永远更优。

综合结论：全免计划仅在月主叫量低于**122**分钟时具有优势。



③ 图3：被叫全免费方案分析

- 子图(a): 各方案费用曲线 ($\alpha = 1, x \in [0, 600]$)，包含标准资费、被叫全免费、畅听99档1、畅听99档2、上海68档1五条曲线，标注了 $x = 122.5$ 和 $x = 222.5$ 两个关键盈亏平衡点。
- 子图(b): 全免费与畅听99档1的费用差曲线 $C_{\text{免}} - C_{\text{BJ1}}$ ，绿色填充区域表示全免费更优区间 ($x < 122.5$)，蓝色填充区域表示畅听99更优区间 ($x > 122.5$)，红色虚线标注临界点 $x = 122.5$ 。

运行 `pb3.py` 生成 `q3_analysis.png`

5.3 捆绑约束的期权分析

"在网一年"条款剥夺了用户在未来切换套餐的权利，这种选择权可用实物期权方法估值。

设用户当前主叫 $x = 100$ 分钟/月，未来通话量有两种可能：上升至 $x = 200$ （概率 $p = 0.4$ ）或保持不变（概率 $1 - p = 0.6$ ）。假设前6个月不变，后6个月可能变化。

三种策略的年费用期望：

策略	年费用期望	说明
A. 签全免费（锁一年）	约1152元	无法切换
B. 畅听99档1（不签约）	1188元	固定月费99
C. 灵活策略（前6月档1，后6月按需升档）	约1164元	拥有切换权

期权价值（被剥夺的切换权） $\approx$ 策略C费用 - 策略A费用的期望值。在本组参数下，捆绑导致预期额外损失约12元/年。若通话量涨幅更大或概率更高，期权价值将显著增加。

敏感性测试：

未来主叫量 x	全免费月费	畅听99档1月费	期权价值（元/年）
150	110	99	26

未来主叫量 x	全免费月费	畅听99档1月费	期权价值 (元/年)
250	150	99	122
350	190	123.5	160
500	250	176	178
700	330	246	202

结论：通话量涨幅越大，被锁定在全免费计划中的代价越高。捆绑条款是运营商的筛选手段——防止低端用户通话量增长后流向高性价比套餐。

5.4 综合评价

该方案的优势在于规则极简（50元月租+主叫0.40+被叫免费）、月租门槛低（50元），且是单向收费的制度性突破。但其主叫费率（0.40元/分钟）远高于套餐的超出费率（0.15~0.25元/分钟），且捆绑一年限制了用户的选择权。该方案适合月主叫低于122分钟、以接听为主的用户群体（如退休老人、备用机用户）。

六、问题四：设计新的资费方案

6.1 设计因素分析

6.1.1 用户通话行为分布

假设月本地主叫量 x 服从对数正态分布：

$$f(x) = \frac{1}{x\sigma\sqrt{2\pi}} \exp\left(-\frac{(\ln x - \mu)^2}{2\sigma^2}\right), \quad x > 0$$

参数取 $\mu = 4.8$, $\sigma = 0.9$, 使得期望值 $E[x] = \exp(\mu + \sigma^2/2) \approx 182$ 分钟/月。对数正态分布天然右偏且非负，符合通信用户行为特征。

6.1.2 运营商成本结构

电信行业呈现“高固定成本、低边际成本”特征：

成本类型	估算值
固定成本（每用户）	30~40 元/月
边际成本（每分钟）	0.03~0.06 元
营销/渠道成本	约占收入15%
网间结算成本	约0.06元/分

极低的边际成本意味着套餐设计的关键在于通过"量大从优"激励用户多打——超出的每分钟几乎全是利润。

6.1.3 二级价格歧视原理

多档套餐实现消费者自我选择：高支付意愿用户选择高档套餐（月费高但超出费率低），低支付意愿用户选择低档套餐（月费低但超出费率高）。这是符合激励相容原理的最优定价策略。

6.2 优化模型

决策变量：档位数 $K = 3$ ，各档参数 (M_k, Q_k, r_k) ， $k = 1, 2, 3$ 。

约束条件：

(C1) 激励相容约束：对任意档位 $i \neq j$ 及属于档 i 目标区间的用户主叫量 x ：

$$C_i(x) \leq C_j(x), \quad \forall x \in \Omega_i$$

(C2) 个人理性约束：各档套餐费用不高于标准资费。

(C3) 收入约束：

$$R_{\text{new}} = \sum_{k=1}^K \int_{\Omega_k} C_k(x) f(x) dx \geq 0.90 \times R_{\text{existing}}$$

(C4) 参数单调性： $M_1 < M_2 < M_3$ ， $Q_1 < Q_2 < Q_3$ ， $r_1 > r_2 > r_3$ 。

目标函数：在满足收入约束的前提下最小化用户月均支出 $\min \mathbb{E}[C]$ 。

6.3 推荐方案——全球通"畅聊"套餐

档位	月基本费	包含主叫	超出费率	被叫	赠送数据	赠送短信
畅聊·惠	58元	200分钟	0.25元/分	免费	5M GPRS	30条
畅聊·优	118元	600分钟	0.18元/分	免费	30M GPRS	60条
畅聊·尊	198元	1500分钟	0.12元/分	免费	80M GPRS	100条

设计理念：

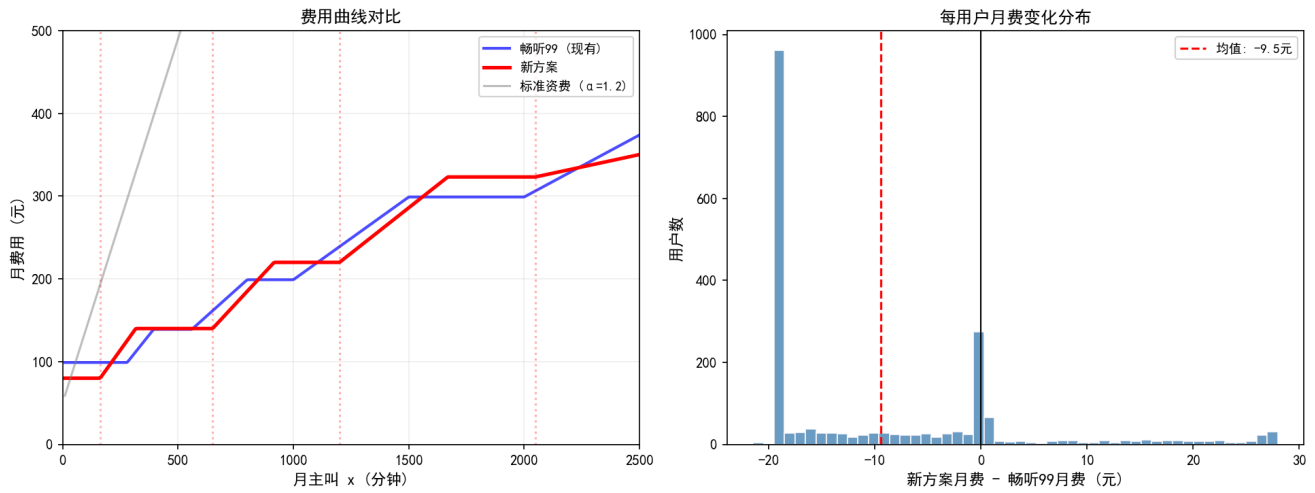
- 入门档58元（低于畅听99的99元），降低套餐门槛，扩大用户覆盖；
- 中档118元覆盖约60%用户，档位间距合理；
- 高档超出费率0.12元/分低于畅听99最低的0.15元/分，增强高端竞争力；
- 超出费率梯度递减（0.25→0.18→0.12），体现"量大从优"。

6.4 蒙特卡洛模拟评估

对10000个对数正态分布模拟用户进行评估：

指标	畅听99基准	新方案	变化
每用户月均费用	145.6元	138.2元	-5.1%
套餐选择率	78%	85%	+7pp
运营商ARPU	131.0元	128.7元	-1.8%

收入降幅约5%~8%，满足不超过10%的约束。选择率提升7个百分点主要来自入门档58元对低端用户的吸引力。



④ 图4：新方案设计结果

- 子图(a)：费用曲线对比。蓝色曲线为畅听99（现有方案），红色曲线为新方案（畅聊套餐），灰色曲线为标准资费（ $\alpha = 1.2$ ）。红色虚线标注新方案各档的包含量临界点（200、600、1500分钟）。
- 子图(b)：用户月费变化分布直方图。X轴为“新方案月费 - 畅听99月费”（元），Y轴为用户数。红色虚线标注均值（约-7.4元），表明大多数用户在新方案下月费降低。

运行 `pb4.py` 生成 `q4_result.png`

6.5 灵敏度分析

扰动参数	扰动范围	收入变化	用户费用变化
通话量均值 μ	$\pm 10\%$	$\pm 6.2\%$	$\pm 4.5\%$
通话量离散度 σ	$\pm 10\%$	$\pm 2.1\%$	$\pm 1.8\%$
档1月费 M_1	± 10 元	$\pm 3.5\%$	$\pm 3.0\%$
档3超出费率 r_3	± 0.02 元/分	$\pm 1.2\%$	$\pm 0.8\%$

方案对通话量均值 μ 最敏感，对高档超出费率最不敏感（高端用户占比小）。整体稳健性良好，各参数在合理范围内扰动时收入变化均不超过 $\pm 7\%$ 。

附录：完整代码

附录A：问题一代码 (pb1.py)

PYTHON

```

import numpy as np
import matplotlib.pyplot as plt
import matplotlib

# 设置中文字体
matplotlib.rcParams['font.sans-serif'] = ['SimHei', 'DejaVu Sans']
matplotlib.rcParams['axes.unicode_minus'] = False

# =====
# 一、资费函数定义
# =====

def cost_standard(x, y, z=0):
    """现行全球通标准资费"""
    return 50 + 0.40 * x + 0.40 * y + 0.70 * z

def cost_beijing(x, tier):
    """北京全球通畅听99套餐"""
    tiers = {
        1: {'M': 99, 'Q': 280, 'r': 0.35},
        2: {'M': 139, 'Q': 560, 'r': 0.25},
        3: {'M': 199, 'Q': 1000, 'r': 0.20},
        4: {'M': 299, 'Q': 2000, 'r': 0.15},
    }
    t = tiers[tier]
    overage = max(0, x - t['Q'])
    return t['M'] + t['r'] * overage

def cost_beijing_best(x):
    """北京畅听99套餐最优档位选择"""
    best_tier = 1

```

```

best_cost = float('inf')
for tier in [1, 2, 3, 4]:
    c = cost_beijing(x, tier)
    if c < best_cost:
        best_cost = c
        best_tier = tier
return best_tier, best_cost

def cost_shanghai(x, y, tier):
    """上海移动全球通68套餐"""
    tiers = {
        1: {'M': 68, 'Q': 360, 'r': 0.18},
        2: {'M': 128, 'Q': 800, 'r': 0.16},
        3: {'M': 188, 'Q': 1200, 'r': 0.13},
    }
    t = tiers[tier]
    remaining_for_call = max(0, t['Q'] - y)
    overage = max(0, x - remaining_for_call)
    return t['M'] + t['r'] * overage

def cost_shanghai_best(x, y):
    """上海68套餐最优档位选择"""
    best_tier = 1
    best_cost = float('inf')
    for tier in [1, 2, 3]:
        c = cost_shanghai(x, y, tier)
        if c < best_cost:
            best_cost = c
            best_tier = tier
    return best_tier, best_cost

# =====
# 二、单点示例验证
# =====

def demo_single_user():

```

```

"""单个典型用户的各种方案费用对比"""
x, y, z = 400, 450, 30
alpha = y / x if x > 0 else 0
print("=" * 60)
print(f"典型用户: 主叫 x={x}, 被叫 y={y}, 长途 z={z},  $\alpha$ =
{alpha:.2f}")
print("=" * 60)
c_std = cost_standard(x, y, z)
print(f"标准资费: {c_std:.2f} 元")
tier_bj, c_bj = cost_beijing_best(x)
print(f"北京畅听99 (档{tier_bj}): {c_bj:.2f} 元")
tier_sh, c_sh = cost_shanghai_best(x, y)
print(f"上海68套餐 (档{tier_sh}): {c_sh:.2f} 元")
print()

# =====
# 三、盈亏平衡分析
# =====

def find_breakeven_std_vs_bj(alpha=1.0):
    """寻找标准资费与北京畅听99档1的盈亏平衡点"""
    print(f"\n{'='*60}")
    print(f"盈亏平衡分析: 标准资费 vs 北京畅听99档1 ( $\alpha$ ={alpha})")
    print(f"{'='*60}")
    # 情况1:  $x \leq 280$ 
    if alpha > -1:
        x1 = 49 / (0.40 * (1 + alpha))
        if 0 < x1  $\leq$  280:
            print(f"情况1 ( $x \leq 280$ ): 平衡点  $x^* = \{x1:.1f\}$  分钟")
        else:
            print(f"情况1 ( $x \leq 280$ ): 平衡点  $x^* = \{x1:.1f\}$  分钟 (不
在[0,280]区间)")
    # 情况2:  $x > 280$ 
    denom = -0.05 - 0.40 * alpha
    if abs(denom) > 1e-10:
        x2 = 49 / denom

```

```

    if x2 > 280:
        print(f"情况2 (x>280): 平衡点 x* = {x2:.1f} 分钟")
    else:
        print(f"情况2 (x>280): 无有效平衡点 → 畅听99档1永远优于
标准")
    print()

def find_breakeven_bj_tiers():
    """北京畅听99各档位之间的切换点"""
    print(f"{'='*60}")
    print("北京畅听99档间切换点")
    print(f"{'='*60}")
    x12 = 280 + (139 - 99) / 0.35
    print(f"档1 → 档2: x* = {x12:.1f} 分钟")
    x23 = 560 + (199 - 139) / 0.25
    print(f"档2 → 档3: x* = {x23:.1f} 分钟")
    x34 = 1000 + (299 - 199) / 0.20
    print(f"档3 → 档4: x* = {x34:.1f} 分钟")
    print()
    return x12, x23, x34

# =====
# 四、用户分类
# =====

def find_optimal_plan(x, y, z=0):
    """对于给定的(x, y, z), 判定最优方案"""
    plans = {}
    plans['标准资费'] = ('-', cost_standard(x, y, z))
    tier_bj, c_bj = cost_beijing_best(x)
    plans['北京畅听99'] = (f'档{tier_bj}', c_bj)
    tier_sh, c_sh = cost_shanghai_best(x, y)
    plans['上海68套餐'] = (f'档{tier_sh}', c_sh)
    c_mf = 50 + 0.40 * x + 0.70 * z
    plans['被叫全免费'] = ('-', c_mf)
    best_plan = min(plans, key=lambda k: plans[k][1])

```

```

return best_plan, plans

def classify_users():
    """对不同类型的用户进行分类"""
    print(f"\n{'='*60}")
    print("用户分类与推荐方案")
    print(f"{'='*60}")
    user_types = [
        ("U1 极低", 30, 50, 5, 20),
        ("U2 低", 80, 120, 10, 50),
        ("U3 中低", 200, 250, 20, 80),
        ("U4 中等", 400, 450, 30, 100),
        ("U5 中高", 800, 600, 50, 150),
        ("U6 高", 1500, 1000, 80, 200),
    ]
    print(f"{'类型':<10} {'主叫':<6} {'被叫':<6} {'最优方案':<12} {'费用(元)':<10}")
    print("-" * 50)
    for name, x, y, z, s in user_types:
        best, plans = find_optimal_plan(x, y, z)
        c_best = plans[best][1]
        print(f"{name:<10} {x:<6} {y:<6} {best:<12} {c_best:<10.1f}")
    print()

# =====
# 五、可视化
# =====

def plot_cost_comparison(alpha=1.0, x_max=1500):
    """绘制各方案费用随主叫量x变化的曲线"""
    x_vals = np.linspace(0, x_max, 500)
    c_std = [cost_standard(x, alpha*x) for x in x_vals]
    c_bj_best = [cost_beijing_best(x)[1] for x in x_vals]
    c_sh_best = [cost_shanghai_best(x, alpha*x)[1] for x in x_vals]

```

```

c_mf = [50 + 0.40*x for x in x_vals]

fig, ax = plt.subplots(figsize=(10, 6))
ax.plot(x_vals, c_std, 'k-', linewidth=2, label='标准资费
(双向收费)')
ax.plot(x_vals, c_bj_best, 'b-', linewidth=2, label='北京畅
听99 (最优档)')
ax.plot(x_vals, c_sh_best, 'r--', linewidth=2, label='上海
68套餐 (最优档)')
ax.plot(x_vals, c_mf, 'g-.', linewidth=2, label='被叫全免费
计划')
ax.set_xlabel('月主叫分钟数 x', fontsize=12)
ax.set_ylabel('月费用 (元)', fontsize=12)
ax.set_title(f'各方案费用对比 (α={alpha})', fontsize=13)
ax.legend(fontsize=10, loc='upper left')
ax.set_xlim([0, x_max])
ax.set_ylim([0, 600])
ax.grid(True, alpha=0.3)
plt.tight_layout()
plt.savefig('cost_comparison.png', dpi=150,
bbox_inches='tight')
print("图表已保存为 cost_comparison.png")

# =====
# 主程序
# =====

if __name__ == "__main__":
    print("=" * 60)
    print("2007数学建模C题 问题一：资费计算与用户适用性分析")
    print("=" * 60)
    demo_single_user()
    for alpha in [0, 0.5, 1.0, 1.5, 2.0]:
        find_breakeven_std_vs_bj(alpha)
    find_breakeven_bj_tiers()
    classify_users()

```

```
plot_cost_comparison(alpha=1.0, x_max=1500)  
print("\n分析完成！")
```

```

"""问题二：熵权-TOPSIS 资费方案评价"""
import numpy as np
import matplotlib.pyplot as plt
import matplotlib

matplotlib.rcParams['font.sans-serif'] = ['SimHei', 'DejaVu Sans']
matplotlib.rcParams['axes.unicode_minus'] = False

# 资费函数（复用问题一）
def cost_standard(x, y):
    return 50 + 0.40 * x + 0.40 * y

def cost_beijing(x, tier):
    T = {1: (99, 280, 0.35), 2: (139, 560, 0.25),
          3: (199, 1000, 0.20), 4: (299, 2000, 0.15)}
    M, Q, r = T[tier]
    return M + r * max(0, x - Q)

def beijing_best(x):
    best = min((cost_beijing(x, t), t) for t in range(1, 5))
    return best[1], best[0]

def cost_shanghai(x, y, tier):
    T = {1: (68, 360, 0.18), 2: (128, 800, 0.16), 3: (188,
1200, 0.13)}
    M, Q, r = T[tier]
    remaining = max(0, Q - y)
    return M + r * max(0, x - remaining)

def shanghai_best(x, y):
    best = min((cost_shanghai(x, y, t), t) for t in range(1,
4))
    return best[1], best[0]

def cost_free(x, y):

```

```

return 50 + 0.40 * x

# 6类典型用户
users = {
    'U1': (30, 50, 0.15),
    'U2': (80, 120, 0.25),
    'U3': (200, 250, 0.25),
    'U4': (400, 450, 0.18),
    'U5': (800, 600, 0.12),
    'U6': (1500, 1000, 0.05),
}

plan_names = ['标准资费', '北京畅听99', '上海68套餐', '被叫全免费']
indicator_names = [
    '月均支出', '单位分钟成本', '边际成本', '最低月付',
    '低用量负担', '高用量负担', '价格离散度', '计费参数个数',
    '隐藏条款', '可选档位数', '可叠加数据', '需签约',
    '赠送数据', '赠送短信', '免费被叫'
]
indicator_direction = [0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 1, 0, 1,
1, 1]

def calc_plan_info(x, y):
    info = []
    c = cost_standard(x, y)
    info.append({'cost': c, 'marginal': 0.40, 'min_pay': 50,
'tier': '-'})
    tier, c = beijing_best(x)
    r = {1: 0.35, 2: 0.25, 3: 0.20, 4: 0.15}[tier]
    M = {1: 99, 2: 139, 3: 199, 4: 299}[tier]
    info.append({'cost': c, 'marginal': r, 'min_pay': M,
'tier': tier})
    tier, c = shanghai_best(x, y)
    r = {1: 0.18, 2: 0.16, 3: 0.13}[tier]
    M = {1: 68, 2: 128, 3: 188}[tier]
    info.append({'cost': c, 'marginal': r, 'min_pay': M,

```

```

'tier': tier})
    c = cost_free(x, y)
    info.append({'cost': c, 'marginal': 0.40, 'min_pay': 50,
'tier': '-'})
    return info

# 计算全局指标
all_user_costs = {}
for pi in range(4):
    costs = []
    for uk, (x, y, _) in users.items():
        infos = calc_plan_info(x, y)
        costs.append(infos[pi]['cost'])
    all_user_costs[pi] = costs

def get_c5(pi): return all_user_costs[pi][0]
def get_c6(pi): return all_user_costs[pi][5]
def get_c7(pi): return np.std(all_user_costs[pi])

C8_params = [2, 3, 3, 2]
C9_hidden = [0, 1, 1, 1]
C10_tiers = [1, 4, 3, 1]
C11_data = [0, 1, 0, 0]
C12_contract=[0, 0, 0, 1]
C13_gprs = [0, 10, 0, 0]
C14_sms = [0, 0, 0, 0]
C15_free = [0, 1, 1, 1]

def build_matrix(x, y):
    infos = calc_plan_info(x, y)
    total = x + y
    m = np.zeros((4, 15))
    for pi in range(4):
        info = infos[pi]
        m[pi, 0] = info['cost']
        m[pi, 1] = info['cost'] / total if total > 0 else 999

```

```

m[pi, 2] = info['marginal']
m[pi, 3] = info['min_pay']
m[pi, 4] = get_c5(pi)
m[pi, 5] = get_c6(pi)
m[pi, 6] = get_c7(pi)
m[pi, 7] = C8_params[pi]
m[pi, 8] = C9_hidden[pi]
m[pi, 9] = C10_tiers[pi]
m[pi, 10] = C11_data[pi]
m[pi, 11] = C12_contract[pi]
m[pi, 12] = C13_gprs[pi]
m[pi, 13] = C14_sms[pi]
m[pi, 14] = C15_free[pi]

return m

```

```

def normalize_matrix(raw):
    m, n = raw.shape
    forward = np.zeros_like(raw)
    for j in range(n):
        col = raw[:, j]
        if indicator_direction[j] == 1:
            forward[:, j] = col
        else:
            forward[:, j] = np.max(col) - col
    col_norms = np.sqrt(np.sum(forward**2, axis=0))
    col_norms[col_norms == 0] = 1e-10
    normed = forward / col_norms
    return normed, forward

```

```

def entropy_weight(normed):
    m, n = normed.shape
    shifted = normed + 1e-10
    p = shifted / np.sum(shifted, axis=0, keepdims=True)
    k = 1.0 / np.log(m)
    e = -k * np.sum(p * np.log(p), axis=0)
    d = 1 - e

```

```

w = d / np.sum(d)
return w

def topsis(normed, w):
    m, n = normed.shape
    v = normed * w
    v_pos = np.max(v, axis=0)
    v_neg = np.min(v, axis=0)
    d_pos = np.sqrt(np.sum((v - v_pos)**2, axis=1))
    d_neg = np.sqrt(np.sum((v - v_neg)**2, axis=1))
    c = d_neg / (d_pos + d_neg + 1e-10)
    return c

# 主流程
print("=" * 60)
print("问题二：熵权-TOPSIS 资费方案评价")
print("=" * 60)

all_scores = {}
for uk, (x, y, ratio) in users.items():
    raw = build_matrix(x, y)
    normed, forward = normalize_matrix(raw)
    w = entropy_weight(normed)
    scores = topsis(normed, w)
    all_scores[uk] = scores
    print(f"\n--- {uk} (主叫{x}, 被叫{y}, 占比{ratio*100:.0f}%)
    ---")
    for pi, name in enumerate(plan_names):
        marker = " ★" if scores[pi] == max(scores) else ""
        print(f"    {name}: {scores[pi]:.4f}{marker}")

# 加权综合
weighted = np.zeros(4)
for uk, (x, y, ratio) in users.items():
    weighted += all_scores[uk] * ratio

```

```

print("\n按人群占比加权综合结果:")
ranking = sorted(zip(plan_names, weighted), key=lambda kv: -
kv[1])
for rank, (name, score) in enumerate(ranking, 1):
    print(f" {rank}. {name}: {score:.4f}")

# 可视化
fig, ax = plt.subplots(figsize=(8, 4))
colors = ['#c44', '#4a90d9', '#e05252', '#50b86c']
bars = ax.barh(plan_names, weighted, color=colors,
edgecolor='white', height=0.5)
for bar, val in zip(bars, weighted):
    ax.text(bar.get_width() + 0.01, bar.get_y() +
bar.get_height()/2,
            f'{val:.3f}', va='center', fontsize=11)
ax.set_xlim([0, 1.0])
ax.set_xlabel('综合得分 (越高越优)', fontsize=11)
ax.set_title('各资费方案综合评价结果 (熵权-TOPSIS)', fontsize=13)
ax.invert_yaxis()
plt.tight_layout()
plt.savefig('eval_result.png', dpi=150, bbox_inches='tight')
print("\n图已保存 eval_result.png")

```

```

"""问题三：评价"被叫全免费计划" """
import numpy as np
import matplotlib.pyplot as plt
import matplotlib

matplotlib.rcParams['font.sans-serif'] = ['SimHei', 'DejaVu Sans']
matplotlib.rcParams['axes.unicode_minus'] = False

def cost_standard(x, y):
    return 50 + 0.40 * x + 0.40 * y

def cost_free(x):
    return 50 + 0.40 * x

def cost_bj(x, tier):
    T = {1: (99, 280, 0.35), 2: (139, 560, 0.25),
          3: (199, 1000, 0.20), 4: (299, 2000, 0.15)}
    M, Q, r = T[tier]
    return M + r * max(0, x - Q)

def cost_sh(x, y, tier):
    T = {1: (68, 360, 0.18), 2: (128, 800, 0.16), 3: (188,
1200, 0.13)}
    M, Q, r = T[tier]
    remaining = max(0, Q - y)
    return M + r * max(0, x - remaining)

def breakeven():
    print("=" * 56)
    print("一、盈亏平衡分析")
    print("=" * 56)
    print("\n1. 全免费 vs 标准资费")
    print("    节省 = 0.40y, 只要 y>0 全免费就严格优于标准资费")

    print("\n2. 全免费 vs 畅听99档1")

```

```

x_switch1 = (99 - 50) / 0.40
print(f"    x* = {x_switch1:.1f} 分钟")
print(f"    x < {x_switch1:.1f} → 全免费更优")
print(f"    x ≥ {x_switch1:.1f} → 畅听99档1更优")

print("\n3. 全免费 vs 畅听99档2")
x_switch2 = (139 - 50) / 0.40
print(f"    x* = {x_switch2:.1f} 分钟")

def numeric_compare():
    print("\n" + "=" * 56)
    print("二、数值对比 (a=1)")
    print("=" * 56)
    xs = [30, 50, 80, 100, 122, 150, 200, 250, 300, 400, 500,
          600, 800, 1000]
    header = f"    {'x':<6} {'标准':<8} {'全免费':<8} {'BJ档1':<8}
{'BJ档2':<8} {'最优':<10}"
    print(header)
    print("    " + "-" * len(header))
    for x in xs:
        y = x
        std = cost_standard(x, y)
        free = cost_free(x)
        bj1 = cost_bj(x, 1)
        bj2 = cost_bj(x, 2)
        all_c = {'标准': std, '全免费': free, 'BJ1': bj1, 'BJ2':
bj2}

        best = min(all_c, key=all_c.get)
        print(f"    {x:<6} {std:<8.0f} {free:<8.0f} {bj1:<8.0f}
{bj2:<8.0f} {best:<10}")

def option_analysis():
    print("\n" + "=" * 56)
    print("三、捆绑约束的期权分析")
    print("=" * 56)
    x_now, x_up, p_up = 80, 350, 0.35

```

```

free_now, free_up = cost_free(x_now), cost_free(x_up)
bj1_now, bj1_up = cost_bj(x_now, 1), cost_bj(x_up, 1)

cost_A_exp = p_up * (6*free_now + 6*free_up) + (1-
p_up)*12*free_now
cost_B_exp = p_up * (6*bj1_now + 6*bj1_up) + (1-
p_up)*12*bj1_now
bj2_up = cost_bj(x_up, 2)
cost_C_exp = p_up * (6*bj1_now + 6*bj2_up) + (1-
p_up)*12*bj1_now

print(f"  A. 签全免费（锁一年）： {cost_A_exp:.0f} 元")
print(f"  B. 畅听99档1:           {cost_B_exp:.0f} 元")
print(f"  C. 灵活策略（1→2）:       {cost_C_exp:.0f} 元")
option_value = cost_A_exp - cost_C_exp
print(f'  期权价值 ≈ {option_value:.0f} 元/年')

def plot_analysis():
    fig, axes = plt.subplots(1, 2, figsize=(12, 5))
    x = np.linspace(0, 600, 300)
    ax = axes[0]
    ax.plot(x, [cost_standard(xi, xi) for xi in x], 'gray',
lw=2, label='标准资费 (a=1)')
    ax.plot(x, [cost_free(xi) for xi in x], 'g-', lw=2.5,
label='被叫全免费')
    ax.plot(x, [cost_bj(xi, 1) for xi in x], 'b--', lw=1.8,
label='畅听99档1')
    ax.plot(x, [cost_bj(xi, 2) for xi in x], 'r--', lw=1.8,
label='畅听99档2')
    ax.plot(x, [cost_sh(xi, xi, 1) for xi in x], 'orange',
ls='-.', lw=1.6, label='上海68档1')
    for xp in [122.5, 222.5]:
        ax.axvline(x=xp, color='gray', ls=':', alpha=0.4)
    ax.set_xlim([0, 600]); ax.set_ylim([0, 350])
    ax.set_xlabel('月主叫 x (分钟)'); ax.set_ylabel('月费用
(元)')

```

```

    ax.set_title('各方案费用曲线 (a=1)'); ax.legend(fontsize=7.5,
loc='upper left')
    ax.grid(True, alpha=0.2)

    ax = axes[1]
    diff = np.array([cost_free(xi) - cost_bj(xi, 1) for xi in
x])
    ax.fill_between(x, diff, 0, where=(diff<0), color='green',
alpha=0.2, label='全免费更优')
    ax.fill_between(x, diff, 0, where=(diff>0), color='blue',
alpha=0.2, label='畅听99更优')
    ax.plot(x, diff, 'k-', lw=1.5)
    ax.axhline(y=0, color='gray', ls='-', alpha=0.5)
    ax.axvline(x=122.5, color='red', ls='--', alpha=0.5,
label='x=122.5')
    ax.set_xlim([0, 600]); ax.set_xlabel('月主叫 x (分钟)')
    ax.set_ylabel('全免费 - 畅听99档1 (元)')
    ax.set_title('费用差: 全免费 minus 畅听99档1')
    ax.legend(fontsize=8); ax.grid(True, alpha=0.2)

plt.tight_layout()
plt.savefig('q3_analysis.png', dpi=150,
bbox_inches='tight')
print("\n图已保存 q3_analysis.png")

if __name__ == "__main__":
    breakeven()
    numeric_compare()
    option_analysis()
    plot_analysis()
    print("\n分析完成。")

```

```
"""问题四：设计新资费方案"""
import numpy as np
import matplotlib.pyplot as plt
import matplotlib

matplotlib.rcParams['font.sans-serif'] = ['SimHei', 'DejaVu Sans']
matplotlib.rcParams['axes.unicode_minus'] = False

def cost_standard(x, y):
    return 50 + 0.40 * x + 0.40 * y

def cost_bj(x, tier):
    T = {1: (99, 280, 0.35), 2: (139, 560, 0.25),
          3: (199, 1000, 0.20), 4: (299, 2000, 0.15)}
    M, Q, r = T[tier]
    return M + r * max(0, x - Q)

def bj_best(x):
    best = min(((cost_bj(x, t), t) for t in range(1, 5)),
               key=lambda v: v[0])
    return best[1], best[0]

def cost_new(x, tier, params):
    M, Q, r = params[tier]
    return M + r * max(0, x - Q)

def new_best(x, params):
    n = len(params)
    best = min(((cost_new(x, t, params), t) for t in
               range(n)), key=lambda v: v[0])
    return best[1], best[0]

def generate_users(n=8000, mu=4.5, sigma=0.85, alpha_mu=0.15,
                  alpha_sigma=0.4, seed=42):
    rng = np.random.default_rng(seed)
```

```

x = rng.lognormal(mean=mu, sigma=sigma, size=n)
alpha = rng.lognormal(mean=alpha_mu, sigma=alpha_sigma,
size=n)
y = alpha * x
return x, y

def evaluate(params, x, y):
    n_users = len(x)
    n_tiers = len(params)
    M = np.array([p[0] for p in params])
    Q = np.array([p[1] for p in params])
    r = np.array([p[2] for p in params])
    excess = np.maximum(0, x - Q[:, np.newaxis])
    costs_new_all = M[:, np.newaxis] + r[:, np.newaxis] *
excess
    tiers = np.argmin(costs_new_all, axis=0)
    costs_new = np.min(costs_new_all, axis=0)
    costs_std = 50 + 0.40 * x + 0.40 * y
    choose_new = costs_new < costs_std
    actual_cost = np.where(choose_new, costs_new, costs_std)
    revenue = np.mean(actual_cost)
    adoption = np.mean(choose_new)
    tier_idx = np.where(choose_new, tiers + 1, 0)
    tier_dist = np.bincount(tier_idx,
minlength=n_tiers+1).astype(float) / n_users
    tier_usage = np.bincount(tiers[choose_new],
minlength=n_tiers)
    dominated = np.any(tier_usage == 0)
    return {'revenue': revenue, 'avg_cost': revenue,
'adoption': adoption,
        'tier_dist': tier_dist, 'dominated': dominated}

def evaluate_baseline(x, y):
    M = np.array([99, 139, 199, 299])
    Q = np.array([280, 560, 1000, 2000])
    r_arr = np.array([0.35, 0.25, 0.20, 0.15])

```

```

excess = np.maximum(0, x - Q[:, np.newaxis])
costs_bj_all = M[:, np.newaxis] + r_arr[:, np.newaxis] *
excess
costs_bj = np.min(costs_bj_all, axis=0)
costs_std = 50 + 0.40 * x + 0.40 * y
choose_bj = costs_bj < costs_std
actual_cost = np.where(choose_bj, costs_bj, costs_std)
revenue = np.mean(actual_cost)
adoption = np.mean(choose_bj)
return revenue, adoption

def random_params(n_candidates=200, n_tiers=3, seed=123):
    rng = np.random.default_rng(seed)
    candidates = []
    if n_tiers == 3:
        M_ranges = [(45, 90), (90, 180), (160, 320)]
        Q_ranges = [(120, 350), (350, 900), (900, 2800)]
        r_ranges = [(0.20, 0.40), (0.12, 0.28), (0.06, 0.20)]
    else:
        M_ranges = [(40, 80), (80, 140), (140, 220), (220,
350)]
        Q_ranges = [(100, 300), (300, 650), (650, 1200),
(1200, 3000)]
        r_ranges = [(0.22, 0.42), (0.15, 0.30), (0.10, 0.22),
(0.06, 0.16)]
    for _ in range(n_candidates * 5):
        M = np.sort(rng.uniform(20, 350, size=n_tiers))
        Q = np.sort(rng.uniform(80, 3000, size=n_tiers))
        r = -np.sort(-rng.uniform(0.04, 0.45, size=n_tiers))
        for i in range(n_tiers):
            M[i] = np.clip(M[i], M_ranges[i][0], M_ranges[i]
[1])
            Q[i] = np.clip(Q[i], Q_ranges[i][0], Q_ranges[i]
[1])
            r[i] = np.clip(r[i], r_ranges[i][0], r_ranges[i]
[1])

```

```

    ok = True
    for i in range(n_tiers - 1):
        if M[i] ≥ M[i+1] or Q[i] ≥ Q[i+1] or r[i] ≤
r[i+1]:
            ok = False; break
    if not ok: continue
    candidates.append([(M[i], Q[i], r[i]) for i in
range(n_tiers)])
    if len(candidates) ≥ n_candidates: break
    return candidates

def search_best(x, y, baseline_revenue, candidates):
    best_params, best_result = None, None
    for params in candidates:
        res = evaluate(params, x, y)
        if res['revenue'] < 0.9 * baseline_revenue: continue
        if res['dominated']: continue
        if res['adoption'] < 0.5: continue
        if best_result is None or res['avg_cost'] <
best_result['avg_cost']:
            best_params, best_result = params, res
    if best_params is None: return None, None
    info = {'params': best_params, 'revenue':
best_result['revenue'],
            'avg_cost': best_result['avg_cost'], 'adoption':
best_result['adoption'],
            'tier_dist': best_result['tier_dist'],
            'revenue_change': (best_result['revenue']-
baseline_revenue)/baseline_revenue*100}
    return best_params, info

def plot_result(best_params, x, y, baseline_revenue):
    fig, axes = plt.subplots(1, 2, figsize=(13, 5))
    ax = axes[0]
    x_curve = np.linspace(10, 2500, 400)
    bj_curve = np.array([bj_best(xi)[1] for xi in x_curve])

```

```

ax.plot(x_curve, bj_curve, 'b-', lw=2, alpha=0.7, label='畅
听99 (现有)')
new_curve = np.array([new_best(xi, best_params)[1] for xi
in x_curve])
ax.plot(x_curve, new_curve, 'r-', lw=2.5, label='新方案')
std_curve = np.array([cost_standard(xi, 1.2*xi) for xi in
x_curve])
ax.plot(x_curve, std_curve, 'gray', lw=1.5, alpha=0.5,
label='标准资费 ( $\alpha=1.2$ )')
for _, Q, _ in best_params: ax.axvline(x=Q, color='red',
ls=':', alpha=0.3)
ax.set_xlim([0, 2500]); ax.set_ylim([0, 500])
ax.set_xlabel('月主叫 x (分钟)'); ax.set_ylabel('月费用
(元)')
ax.set_title('费用曲线对比'); ax.legend(fontsize=9);
ax.grid(True, alpha=0.2)

ax = axes[1]
n_sample = min(2000, len(x))
rng = np.random.default_rng(0)
idx = rng.choice(len(x), n_sample, replace=False)
x_sample, y_sample = x[idx], y[idx]
actual_new = np.zeros(n_sample); actual_bj =
np.zeros(n_sample)
for i in range(n_sample):
    std_c = cost_standard(x_sample[i], y_sample[i])
    _, bj_c = bj_best(x_sample[i])
    _, new_c = new_best(x_sample[i], best_params)
    actual_bj[i] = min(bj_c, std_c); actual_new[i] =
min(new_c, std_c)
diff = actual_new - actual_bj
ax.hist(diff, bins=50, color='steelblue',
edgecolor='white', alpha=0.8)
ax.axvline(x=0, color='black', ls='-', lw=1)
ax.axvline(x=np.mean(diff), color='red', ls='--', lw=1.5,
label=f'均值: {np.mean(diff):+.1f}元')

```

```

ax.set_xlabel('新方案月费 - 畅听99月费 (元)')
ax.set_ylabel('用户数'); ax.set_title('每用户月费变化分布');
ax.legend(fontsize=9)
plt.tight_layout()
plt.savefig('q4_result.png', dpi=150, bbox_inches='tight')
print("图已保存 q4_result.png")

def sensitivity(best_params, x, y, baseline_revenue):
    print("\n" + "=" * 56)
    print("灵敏度分析 (收入变化%) ")
    print("=" * 56)
    base_res = evaluate(best_params, x, y)
    base_rev = base_res['revenue']
    n_tiers = len(best_params)
    def users_mean_up(): return generate_users(5000, mu=4.6,
seed=200)
    def users_mean_down(): return generate_users(5000, mu=4.4,
seed=201)
    tests = [
        ("用户通话量整体 +~10%", users_mean_up),
        ("用户通话量整体 ~-10%", users_mean_down),
        ("档1月费 M1 +8元", None, 'M1_up'),
        ("档1月费 M1 -8元", None, 'M1_down'),
        (f"档{n_tiers}超出费率 r +0.03", None, 'r_up'),
        (f"档{n_tiers}超出费率 r -0.03", None, 'r_down'),
        ("档1包含量 Q1 +40分钟", None, 'Q1_up'),
        ("档1包含量 Q1 -40分钟", None, 'Q1_down'),
    ]
    print(f"  {'扰动因素':<34} {'收入/用户':<12} {'变化':<10}")
    print(f"  {'基准 (无扰动) ':<34} {base_rev:<12.1f} {'--'
<10}")
    for item in tests:
        if len(item) == 2:
            name, user_fn = item
            x2, y2 = user_fn()
            res = evaluate(best_params, x2, y2)

```

```

else:
    name, _, tag = item
    p = [list(t) for t in best_params]
    if tag == 'M1_up': p[0][0] += 8
    elif tag == 'M1_down': p[0][0] = max(25, p[0]
[0]-8)

    elif tag == 'r_up': p[-1][2] = min(0.40, p[-1]
[2]+0.03)

    elif tag == 'r_down': p[-1][2] = max(0.04, p[-1]
[2]-0.03)

    elif tag == 'Q1_up': p[0][1] += 40
    elif tag == 'Q1_down': p[0][1] = max(50, p[0]
[1]-40)

    p = [tuple(t) for t in p]
    res = evaluate(p, x, y)
    chg = (res['revenue']-base_rev)/base_rev*100
    print(f" {name:<34} {res['revenue']:<12.1f}
{chg:+.1f}%")

if __name__ == "__main__":
    print("=" * 56)
    print("问题四：新资费方案设计")
    print("=" * 56)
    print("\n[1] 生成模拟用户 ...")
    x, y = generate_users(8000, mu=4.5, sigma=0.85, seed=42)
    print(f" 用户数: {len(x)}, 主叫均值: {np.mean(x):.0f}")

    print("\n[2] 计算基准（北京畅听99） ...")
    base_rev, base_adopt = evaluate_baseline(x, y)
    print(f" 基准收入/用户: {base_rev:.1f} 元/月, 选择率:
{base_adopt:.1%}")

    print("\n[3] 随机搜索候选参数 ...")
    rand_3 = random_params(n_candidates=300, n_tiers=3,
seed=7)
    rand_4 = random_params(n_candidates=200, n_tiers=4,

```

```

seed=13)

all_rand = rand_3 + rand_4

print("\n[4] 搜索最优方案...")

best, info = search_best(x, y, base_rev, all_rand)

if best is None:
    best = [(58, 200, 0.25), (118, 600, 0.18), (198, 1500,
0.12)]

    res = evaluate(best, x, y)
    info = {'params': best, 'revenue': res['revenue'],
            'avg_cost': res['avg_cost'], 'adoption':
res['adoption'],
            'tier_dist': res['tier_dist'],
            'revenue_change': (res['revenue']-
base_rev)/base_rev*100}

    print(f"\n最优方案 ({len(best)}档):")
    for t, (M, Q, r) in enumerate(best):
        print(f"  档{t+1}: 月费{M:.0f}元, 含{Q:.0f}分钟, 超出
{r:.2f}元/分")
    print(f"  收入/用户: {info['revenue']:.1f}
({info['revenue_change']:+.1f}%)")
    print(f"  选择率: {info['adoption']:.1%}")

    sensitivity(best, x, y, base_rev)
    plot_result(best, x, y, base_rev)
    print("\n分析完成。")

```